```java
/**
 * A simple Higher-or-Lower card guessing game.
 * Players guess whether the next card will be higher, lower, or equal.
 * Freeze tokens allow skipping a card, and equal guesses give bonus
points.
 * The game ends on an incorrect guess or when the deck is empty.
 */

import java.util.ArrayList;
import java.util.Collections;
import java.util.Random;
import java.util.Scanner;

public class Main {

    private static ArrayList<Integer> deck = new ArrayList<>();
    private static Random rand = new Random();
    private static int freezeTokens = 2;
    private static int score = 0;
    private static int currentCard;

   /**
     * The main entry point for the Higher or Lower game.
     * Initializes the deck, shuffles it, handles game flow and user
interaction.
     */

    public static void main(String[] args) {

        Scanner scnr = new Scanner(System.in);

        initializeDeck();
        shuffleDeck();

        printGameIntro();

        currentCard = drawCard();
        System.out.println("Your starting card is: " +
cardName(currentCard));
```

```
        while (!deck.isEmpty()) {

            printStatus();

            System.out.print("Guess (h = higher, l = lower, e = equal, f =
freeze): ");
            char choice = scnr.next().toLowerCase().charAt(0);

            if (choice == 'f') {
                processFreeze();
                continue;
            }

            int nextCard = drawCard();
            System.out.println("Next card drawn: " + cardName(nextCard));

            boolean stillPlaying = processGuess(choice, nextCard);

            if (!stillPlaying) {
                break;
            }

            currentCard = nextCard;

        }

        System.out.println("Game Over");
        System.out.println("Final score: " + score);
        System.out.println("Thanks for playing");

        scnr.close();

  /**
     * Initializes the deck with four copies of each card value (1-13).
     */

    }

    private static void initializeDeck() {
        deck.clear();
```

```
        for (int value = 1; value <= 13; value++) {
            for (int i = 0; i < 4; i++) {
                deck.add(value);
            }
        }
    }

/**
     * Randomly shuffles the deck.
     */


    private static void shuffleDeck() {
        Collections.shuffle(deck);
    }

/**
     * Draws the top card from the deck.
     *
     * @return the integer card value drawn
     */

    private static int drawCard() {
        return deck.remove(0);
    }

/**
     * Converts a card's integer value to a readable name.
     *
     * @param value numeric card value
     * @return the card name (e.g., Ace, King, 5)
     */

    private static String cardName(int value) {
        switch (value) {
            case 1: return "Ace";
            case 11: return "Jack";
            case 12: return "Queen";
            case 13: return "King";
            default: return String.valueOf(value);
```

```
        }
    }

   /**
     * Prints the game introduction and rules.
     */

    private static void printGameIntro() {
        System.out.println("Higher or Lower: Freeze and Equal Bonus");
        System.out.println("Rules");
        System.out.println("You start with 2 freeze tokens. Use (f) to
skip to next card");
        System.out.println("Guess higher(h) or lower(l) to earn +1
point.");
        System.out.println("You may guess equal(e). If correct, +3 points!
If not, -2 points");
        System.out.println("The game ends when you guess incorrectly or
the deck is empty.");
    }

  /**
     * Prints the current game status including score, remaining cards,
and freezes.
     */

    private static void printStatus() {
        System.out.println("Current card: " + cardName(currentCard));
        System.out.println("Score: " + score);
        System.out.println("Freeze tokens left: " + freezeTokens);
        System.out.println("Cards remaining: " + deck.size());
    }

  /**
     * Processes the player's guess and determines whether the game
continues.
     *
     * @param guess the player's guess (h, l, or e)
     * @param nextCard the card drawn after the guess
     * @return true if the player may continue, false if they lose
     */
```

```
    private static boolean processGuess(char guess, int nextCard) {
        switch (guess) {
            case 'h': return handleHigherGuess(nextCard);
            case 'l': return handleLowerGuess(nextCard);
            case 'e': return handleEqualGuess(nextCard);
            default:
                System.out.println("Invalid input. Try again.");
                return true;
            }
        }

    /**
     * Handles a higher guess.
     *
     * @param nextCard the card drawn
     * @return true if correct, false if incorrect
     */

    private static boolean handleHigherGuess(int nextCard) {
        if (nextCard > currentCard) {
            System.out.println("Correct! It was higher.");
            score = score + 1;
            return true;
        } else {
            System.out.println("Incorrect! It was not higher");
            return false;
        }
    }

/**
     * Handles a lower guess.
     *
     * @param nextCard the card drawn
     * @return true if correct, false if incorrect
     */

    private static boolean handleLowerGuess(int nextCard) {
        if (nextCard < currentCard) {
            System.out.println("Correct! It was lower.");
            score = score + 1;
```

```
            return true;
        } else {
            System.out.println("Incorrect! It was not lower.");
            return false;
        }
    }

/**
     * Handles an equal guess.
     *
     * @param nextCard the card drawn
     * @return true if correct, false if incorrect
     */

    private static boolean handleEqualGuess(int nextCard) {
        if (nextCard == currentCard) {
            System.out.println("Incredible! You guessed equal!");
            score = score + 3;
            return true;
        } else {
            System.out.println("Wrong! Equal was incorrect.");
            score = score - 2;
            if (score < 0) {
                score = 0;
            }
            return false;
        }
    }

   /**
     * Processes the freeze action.
     * Uses a freeze token if available, draws a new card, and skips the
guess.
     */

    private static void processFreeze() {
        if (freezeTokens > 0) {
            freezeTokens = freezeTokens - 1;
            currentCard = drawCard();
```

```
            System.out.println("You used a freeze token. Skipping and
drawing a new card.");
            System.out.println("New card is: " + cardName(currentCard));
        } else {
            System.out.println("You have no freeze tokens left.");
        }
    }
}
```